Лабораторная работа №3
Зусь Александра, гр. 4404

Задание: Напишите программу, которая читает из файла координаты точек в 3D пространстве (x,y,z) и вычисляет геометрический центр, который есть среднее по x, y и z.

Напишите две версии программы: одну с расспараллеливанием цикла, другую с функциональной декомпозицией.

Код программы:

```
#include <iostream>#include <fstream>#include <vector>#include <string>#include <chrono>#include <omp.h>#include
<cstdlib>#include <ctime>#include <iomanip>
using namespace std;
struct Point {
    double x, y, z;
};
// ====================== Генерация файла ======================
void generate_points(const string& filename, int count) {
    ofstream out(filename);
    if (!out) {
        cerr << "Cannot create file " << filename << endl;
        return;
    }

    srand(time(nullptr));

    for (int i = 0; i < count; ++i) {
        double x = (rand() % 20000 - 10000) / 100.0;
        double y = (rand() % 20000 - 10000) / 100.0;
        double z = (rand() % 20000 - 10000) / 100.0;
        out << x << " " << y << " " << z << "\n";
    }

    cout << "File \"" << filename << "\" created with " << count << " points.\n";
}
// ====================== Чтение точек ======================bool read_points(const string& filename,
vector<Point>& points) {
    ifstream in(filename);
    if (!in) return false;

    double x, y, z;
    while (in >> x >> y >> z) {
        points.push_back({ x, y, z });
    }
    return !points.empty();
}
// ====================== Версия 1: распараллеливание цикла ======================void centroid_loop(const
vector<Point>& points, double& cx, double& cy, double& cz) {
    double sx = 0.0, sy = 0.0, sz = 0.0;
    int n = (int)points.size();
#pragma omp parallel for reduction(+:sx, sy, sz)
    for (int i = 0; i < n; ++i) {
        sx += points[i].x;
        sy += points[i].y;
        sz += points[i].z;
    }

    cx = sx / n;
    cy = sy / n;
    cz = sz / n;
}
// ====================== Версия 2: функциональная декомпозиция ======================void
centroid_functional(const vector<Point>& points, double& cx, double& cy, double& cz) {
    double sx = 0.0, sy = 0.0, sz = 0.0;
    int n = (int)points.size();
```

```
#pragma omp parallel sections
    {#pragma omp section
        {for (int i = 0; i < n; ++i) sx += points[i].x;}#pragma omp section
        {for (int i = 0; i < n; ++i) sy += points[i].y;}#pragma omp section
        {for (int i = 0; i < n; ++i) sz += points[i].z;}
    }

    cx = sx / n;
    cy = sy / n;
    cz = sz / n;
}
// ====================== main ====================
int main(int argc, char* argv[]) {
    // Режим генерации файла
    if (argc >= 2 && string(argv[1]) == "generate") {
        int count = (argc >= 3) ? atoi(argv[2]) : 1000000;
        generate_points("points.txt", count);
        return 0;
    }

    if (argc != 3) {
        cerr << "Usage:\n";
        cerr << "  Generate:  " << argv[0] << " generate [count]\n";
        cerr << "  Calculate: " << argv[0] << " <points_file> <num_threads>\n";
        return 1;
    }

    string filename = argv[1];
    int threads = atoi(argv[2]);
    omp_set_num_threads(threads);

    vector<Point> points;
    if (!read_points(filename, points)) {
        cerr << "Cannot read points from " << filename << endl;
        return 1;
    }

    cout << "Points loaded: " << points.size() << endl;
    cout << "Threads: " << threads << endl << endl;

    double cx, cy, cz;

    // ----- Версия 1: цикл -----
    auto start = chrono::high_resolution_clock::now();
    centroid_loop(points, cx, cy, cz);
    auto end = chrono::high_resolution_clock::now();
    double t1 = chrono::duration<double>(end - start).count();

    cout << fixed << setprecision(6);
    cout << "=== Loop parallelization (reduction) ===\n";
    cout << "Center: (" << cx << ", " << cy << ", " << cz << ")\n";
    cout << "Time:   " << t1 << " s\n\n";

    // ----- Версия 2: функциональная декомпозиция -----
    start = chrono::high_resolution_clock::now();
    centroid_functional(points, cx, cy, cz);
    end = chrono::high_resolution_clock::now();
    double t2 = chrono::duration<double>(end - start).count();

    cout << "=== Functional decomposition (sections) ===\n";
    cout << "Center: (" << cx << ", " << cy << ", " << cz << ")\n";
    cout << "Time:   " << t2 << " s\n";

    return 0;
}
```

```
D:\Code\3laba\centroid\x64\Release>centroid.exe generate 1000000
File "points.txt" created with 1000000 points.
```

Рисунок 1. Работоспособность функции генерации файла с точками.

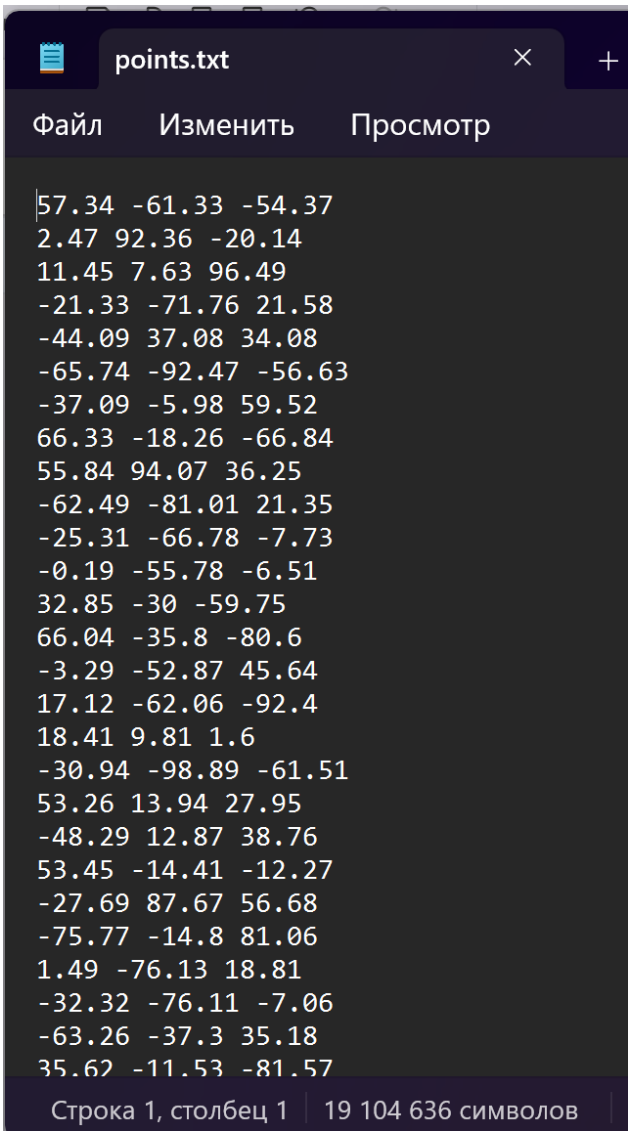


Рисунок 2. Содержание сгенерированного файла «points.txt».

```
D:\Code\3laba\centroid\x64\Release>centroid.exe points.txt 1
Points loaded: 1000000
Threads: 1

=== Loop parallelization (reduction) ===
Center: (-14.067090, -14.054853, -14.049680)
Time:   0.001314 s

=== Functional decomposition (sections) ===
Center: (-14.067090, -14.054853, -14.049680)
Time:   0.002898 s
```

Рисунок 3. Первый запуск: число потоков = 1.

```
D:\Code\3laba\centroid\x64\Release>centroid.exe points.txt 2
Points loaded: 1000000
Threads: 2

=== Loop parallelization (reduction) ===
Center: (-14.067090, -14.054853, -14.049680)
Time:   0.001241 s

=== Functional decomposition (sections) ===
Center: (-14.067090, -14.054853, -14.049680)
Time:   0.003027 s
```

Рисунок 3. Второй запуск: число потоков = 2.

```
D:\Code\3laba\centroid\x64\Release>centroid.exe points.txt 4
Points loaded: 1000000
Threads: 4

=== Loop parallelization (reduction) ===
Center: (-14.067090, -14.054853, -14.049680)
Time:   0.001185 s

=== Functional decomposition (sections) ===
Center: (-14.067090, -14.054853, -14.049680)
Time:   0.002745 s
```

Рисунок 4. Третий запуск: число потоков = 4.

```
D:\Code\3laba\centroid\x64\Release>centroid.exe points.txt 8
Points loaded: 1000000
Threads: 8

=== Loop parallelization (reduction) ===
Center: (-14.067090, -14.054853, -14.049680)
Time:   0.001631 s

=== Functional decomposition (sections) ===
Center: (-14.067090, -14.054853, -14.049680)
Time:   0.002860 s
```

Рисунок 4. Четвёртый запуск: число потоков = 8.

Анализ результатов

Эксперименты проводились на файле, содержащем 1 000 000 точек. Измерялось время работы двух реализаций:

- распараллеливание цикла с использованием **reduction**;
- функциональная декомпозиция с помощью директивы **sections**.

Сравнение времени выполнения:

Число потоков	Распараллеливание цикла	Функциональная декомпозиция
1	0.001314 с	0.002898 с
2	0.001241 с	0.003027 с
4	0.001185 с	0.002745 с
8	0.001631 с	0.002860 с

Анализ распараллеливания цикла (reduction)

Версия с распараллеливанием цикла работает заметно быстрее, чем версия с функциональной декомпозицией. При использовании 2 и 4 потоков время выполнения немного уменьшается. Однако при 8 потоках время снова немного возрастает. Это происходит потому, что на каждую точку приходится очень мало вычислений (только сложение трёх чисел). В результате основная часть времени тратится не на сами вычисления, а на работу с памятью и на создание потоков. Когда потоков становится слишком много, дополнительные затраты на их создание и синхронизацию начинают превышать пользу от параллельной работы.

Анализ версии с функциональной декомпозицией (sections)

В этой реализации используются всего три параллельные секции (подсчёт суммы по X, Y и Z). Поэтому даже при указании 8 потоков реально полезную работу выполняют только три потока. Остальные потоки простаивают, создавая дополнительные накладные расходы. В результате увеличение числа потоков практически не даёт ускорения, а в ряде случаев приводит к небольшому замедлению.

Выводы:

- Распараллеливание цикла с **reduction** является более эффективным способом решения данной задачи.
- Функциональная декомпозиция через **sections** в данном случае ограничена количеством независимых подзадач (всего 3) и поэтому плохо масштабируется.
- При очень маленьком объёме работы на элемент (как в задаче вычисления геометрического центра) ускорение от увеличения числа потоков ограничено и может даже снижаться из-за накладных расходов.